

Closeout of the amplification-checker spike, and the brief for the next one

This document is written for someone who will start the next spike with no other context. It says what was built, how the programs it was judged against were labeled, what the results were and how much each result is worth, and what problems are open, in the order they should be taken up. The artifact itself is described at length, with every report it produced, in `design_docs/2026-09_amplification_checker_overview.md`; this document is the shorter account that points there.

1. What was built

The checker is the function `hydro_lang::sim::amplification::check`. It takes a compiled Hydro simulation, a `CheckConfig`, and a closure that sends one round of a steady workload into the program's inputs, and it returns a `Report`.

It works by perturbing one scheduling decision at a time. Hydro's simulator already exposes every point at which a step of a process decides how many waiting messages to admit (a `use::batch` call, which we call a batch decision point) and every point at which a step decides how old a version of a piece of state to observe (a `use::snapshot` call, a snapshot decision point). The checker first runs the program once under a fixed reference schedule in which every batch point admits everything waiting and every snapshot point sees the newest version, which establishes baseline counts and discovers every decision point that ever had something waiting. Then, for each discovered point, it runs the program again on the same input with that one point held for 1, 2, 4, 8 and so on rounds, doubling up to and including a hold that lasts to the end of the run, while every other decision follows the reference schedule. A held batch point delivers nothing during the hold and everything at once when it ends; a held snapshot point keeps showing the version it saw when the hold began and jumps to the newest on release.

It measures work with two counts collected passively inside the simulator, with no instrumentation of the program: the number of records admitted through batch decision points, summed across the program and kept per cluster member, and the number of network messages each process or member sends. Both counts have the property that a delay alone cannot raise them, because a delay changes when records are admitted and not how many arrive, so a positive difference between a held run and the baseline is work the program derived in response to the delay.

The verdict rule is that a program is hazardous if any hold of any decision point raises either count above the baseline, reading messages per sender and admitted records in total, and benign otherwise. The shape of the curve over hold lengths is reported and is not a criterion. The location rule names, among the points that reacted, the one whose reaction appeared at the shortest hold, breaking ties by the extra work at that length and then by the largest extra work over the curve. The one setting is the horizon, the run length in rounds, with a default of 1000; a benign verdict is printed as "no reaction to a delay of up to N rounds was found at any decision point".

The user-facing entry point is the attribute `#[amplification_check(...)]`, exported from `hydro_lang::sim::amplification` and implemented in the proc-macro crate `hydro_amplification_macro`. Placed on a Hydro function, it generates a test that builds the simulation from the function's signature: `&Process` and `&Cluster` parameters become locations the harness creates, every `Stream<_, _, Unbounded>` parameter becomes an input fed once per round (so timers need nothing), every other parameter is a plain value given by name in the attribute, and generic type parameters are given as `T = u64`. An optional `workload(param = rate)` clause sets per-round counts, a list of rates runs the check once per rate, and a `round = |round, inputs| { ... }` clause replaces the generated workload. A returned struct of streams derives `SimOutputs` so the harness can sink every stream. The script `cargo-check-amplification` at the repository root runs every annotated function in a crate and prints a summary table.

The simulator gained four small additions, each inert unless a hold is set or counting is enabled. Hooks report their source location, item type, and kind, and the location is the line of the program's own `use::batch` or `use::snapshot` call, obtained at macro-expansion time by a new `span_location!` macro in the `copy_span`

crate because rustc collapses debug-info locations of code expanded from an external macro to the call site. A one-clause change to the scheduler lets a step whose only waiting input sits behind a held hook park instead of being forced to admit something. The hold driver pins a held snapshot to its last version and skips to the newest on release. Passive counters record admitted records per hook per member and network messages per sender. The simulator's own test suite passes unchanged.

The entry points by path are: `hydro_lang/src/sim/amplification.rs` (`check`, `CheckConfig`, `Report`), `hydro_lang/src/sim/amplification_harness.rs` (`InputValue`, `SimOutputs`, the results recorder), `hydro_amplification_macro/src/lib.rs` (the attribute and derive), `hydro_lang/src/sim/hold_one_hook.rs` (the hold driver), `hydro_lang/src/sim/work_counts.rs` (the counters), `hydro_lang/src/sim/prompt_schedule.rs` (the reference schedule), `copy_span/src/lib.rs` (`span_location!`), and `cargo-check-amplification` (the runner).

2. The corpus and how labels are established

A label is ground truth for one configuration of one program, set by the program's author before the checker ran. Two bases are accepted. A configuration is hazardous by exhibited collapse when a test on the fixed deterministic schedule runs many rounds of light steady load, applies a bounded trigger, removes it, and asserts in the tail of the run that redundant work persists: a backlog that keeps growing, completions at zero, or a storm still running. A storm that lasts hundreds of rounds after a twenty-round trigger counts. A configuration is benign by assurance argument when its author wrote down why total work is a closed-form function of the input under every schedule and asserted that closed form under a few hundred random schedules and under a burst. A configuration that recovers under one burst is not thereby benign; it has no ground truth. Configurations that carry the mechanism of a collapsing configuration together with a tuning parameter that kept the test's burst from collapsing them are recorded as "no ground truth; tool expected to find amplification", because a tuning parameter can move the threshold at which a mechanism runs away but cannot remove the mechanism.

Authors of witness programs are firewalled from the checker. The brief a fresh session receives is `design_docs/2026-09_blind_witness_task.md`; it embeds the definitions, the label bases, the simulator facts, and the process rules, and it forbids reading any design document, the checker's source, or the checker's test modules. The checker is then run on the new programs by an operator who reads only each function's signature and its steady-state input, and the verdicts are written down before any label is read. The two records of that protocol are `design_docs/2026-09_blind_results.md` and `design_docs/2026-09_blind_budget_results.md`.

There are three sets of programs. The original set is twelve programs under `hydro_test/src/cluster/witnesses/` with `rpc_retry` one directory up, giving twenty-two configurations: request-response retry (`rpc_retry`, three attempts and one), retry with exponential backoff (`backoff_retry`), a bounded and an unbounded server queue (`bounded_queue_rejection`), a read-through cache with a time-to-live in three variants (`cache_thundering_herd`), a log store whose compaction runs on leftover budget (`compaction_falls_behind`), full-state gossip of a grow-only set (`crdt_gossip_load`), heartbeat-timeout election with uniform and staggered timeouts (`election_stampede`), delta gossip with and without re-send until acknowledged (`gossip_resend`), lease renewal with and without re-send (`lease_renewal_storm`), a pure heartbeat (`pure_heartbeat_load`), work-queue rebalancing on stale reports with and without a cooldown (`rebalancing_ping_pong`), and a semi-naïve transitive closure (`transitive_closure_load`). The compaction store's two configurations are excluded from the scorecard, because its author modeled the cost of a read as an integer inside a record rather than as records moved, so no work actually increases in it and a record count is right to see nothing; it should be rewritten to scan real records. The blind set is seven programs under `witnesses/blind/`: a batching sink that retains items until acknowledged (`batch_flush`), a replicated log catch-up protocol (`log_catchup`), two-phase commit with participant timeouts (`two_phase_commit`), a work queue with a visibility timeout (`visibility_queue`), a bounded publish-subscribe fan-out (`bounded_fanout`), credit-based flow control (`credit_flow`), and a token-bucket rate limiter (`token_bucket`). The retry-governance set is five configurations of one service under `witnesses/blind_budget/`, differing only in the client's retry policy: no governance, a time-refilled token budget, a success-refilled token budget, a hybrid budget refilled by both, and a circuit breaker.

3. Results, with their standing

On the original corpus, with the compaction rows excluded, twenty configurations remain: nine hazardous by exhibited collapse, seven benign by assurance argument, and four expectation-only. The checker agrees with all sixteen ground-truth labels and all four expectations, reading nothing but the simulator's counts, and the verdicts are the same at horizons of 240 and 1000 rounds. In every hazardous case the point it names is an edge of the loop the mechanism runs through. This result is in-sample. The verdict rule's particular form, messages per sender and records in total, was chosen after a plain total missed the staggered election and a per-point breakdown wrongly flagged the one-attempt retry service, and the hold sequence and location rule were later changed with the corpus as the only check that verdicts survived, so these twenty agreements measure how well the rule fits the programs it was fitted to.

On the blind set of seven, with the rule frozen and verdicts recorded before labels were read, the checker agreed on all seven, produced no false hazardous verdict on the three benign programs, and on each hazardous program named the point the author's own description places the mechanism at. Two qualifications apply. The operator saw test names that implied the labels while extracting inputs, which could not affect the checker's output but weakened the operator's independence. And all four hazardous programs amplify by the same mechanism as the corpus, a timed re-send whose redundant work crosses a network edge, while the three benign ones never send anything twice, so this set tested independence from the method and not independence from the one hazard shape the method was built around.

On the retry-governance set of five, the checker agreed with three of the author's labels and produced its first false hazardous verdicts, on both configurations the author labeled benign by assurance argument. The success-refilled budget was predicted to go flat, and on two of its three reacting edges it does: holding the replies edge or the server's clock costs exactly the ten initial tokens and no more. On the server's arrivals edge it rises, 21, 37, 69, 133, 261, 498 extra messages at holds of 16 through 512 rounds, about k messages at a hold of k rounds. The mechanism is the release: the dumped backlog drains at three per round with queueing delay above the six-tick timeout for about two thirds of k rounds, every completion during the drain is a successful reply that refills a quarter token, and the client spends each token on a retry whose original is still in the queue. The author's bound of 510 extra sends holds (498 measured at 512), so both statements are true at once: total work is a bounded multiple of the input, and extra work grows with the delay. The circuit breaker reacts once, with sixteen extra records at a hold of eight rounds before it opens, and then does less work than the unheld run at every longer hold, because it drops new input while open; the count reads work done and not work avoided, so the sixteen records make it hazardous. A candidate rule that reads growth across the doublings would recover the breaker and still flag the success budget; a stricter reading that calls any step-and-hold a bounded reaction would also call the ungoverned and time-refilled configurations bounded, since their curves step to a cap of two re-sends per request. No shape rule separates all five. The time-refilled budget's refill rate never bound at the tested workload, so the prediction that it would grow with a smaller slope was not tested.

4. Open problems for the next spike, in priority order

4a. The definition's two clauses conflict

The working definition says a program is hazardous if some schedule makes the same input cost more work, and more the longer delivery is delayed, and benign if work is a function of the input alone under every schedule. The success-refilled budget satisfies the second clause, with total work at most 1.25 times the input plus ten, and violates the first, with extra work rising about linearly in the hold length up to that bound. The checker can see only the first clause, because it holds the input fixed and reads growth against delay; a bound expressed as a multiple of the input is invisible to it by construction. What decided collapse versus recovery in the author's burst tests was the multiplier against the workload: 1.25 times two requests per round fits under a capacity of five, and three times two does not.

The first step is to decide which clause the label should follow, and then to evaluate candidate rules against all thirty-four labeled configurations (twenty scored corpus, seven blind, five retry-governance, plus the two excluded compaction rows for reference). Three candidates are worth evaluating. The report could state

extra work as a multiple of the input alongside the verdict, so a reader sees "up to 1.25 times the input under the worst schedule found" rather than a bare hazardous. The verdict could become three-way: no reaction, bounded reaction with its multiple, and growing reaction, with the boundary defined by whether the curve is still rising at the last hold that ends before the horizon. And the count could read work avoided as well as work done, so that a program that sheds input while it protects itself, as the breaker does, is reported as shedding rather than as amplifying. Whichever is chosen, the retry-governance set is the test, and it is the first set on which the present rule is wrong.

4b. The location rule's weakness

The rule ranks decision points by the shortest hold at which any extra work appears. On the blind program `log_catchup` a leader timer whose reaction is a single extra message at a hold of eight rounds outranks the fetch-arrivals edge whose reaction is 1759 extra records at sixteen, so the report names a point that reacts slightly to a short delay over the point that carries the mechanism. Any fix that ignores small reactions introduces a threshold, and a threshold is a tuning knob of the kind the hold grid was removed for being. The first step is to collect, across all thirty-four configurations, the named point and the point a reader would name from the mechanism, and to look for a rule that agrees on all of them without a numeric threshold; a candidate is to rank by the extra work at the shortest hold at which the point's reaction has become a fixed fraction of its own maximum, which uses only the point's own curve.

4c. Coverage of schedule shapes

The checker searches only single-point contiguous holds beginning at round 1. A hazard that requires two edges to be delayed together, a delay that must begin at a particular moment, a reordering of records on an edge, or delivery of a subset of waiting records is outside the search, and a benign verdict is bounded accordingly. Pairs and triples of holds are affordable: a run costs about two seconds at a horizon of 240, the corpus programs have five to ten decision points, and the doubling sequence gives about eleven lengths per point, so single holds are under two hundred runs per program and take minutes; pairs are up to forty-five point pairs times about a hundred and twenty length pairs, several thousand runs, an hour or two per program at that horizon; triples are tens of thousands of runs, a weekend. Relative start offset matters for pairs and multiplies the count by a small factor. Reordering and subset delivery are a different primitive from delay; the simulator already chooses order and subset on unordered streams, so a driver can hold one member's records or one key's records on such an edge, but on ordered edges the simulator only releases prefixes.

The corpus contains no hazard that needs any of these shapes, and that is the difficulty rather than a comfort. New search shapes are justified by witnesses that need them, and witnesses are written by people who have to think of the hazard first, so the checker's coverage risks being the list of shapes its developers imagined. The first step is an anytime search: run level one exhaustively as now, then level two as a background job, and print in every report which levels completed, so that a benign verdict names its scope. The second step is to ask a blind author for programs whose hazard needs two simultaneous delays or a reordering, without naming the mechanism, and to judge the pair level on them.

4d. The workload

The workload is supplied by the caller. The attribute's default is one record per round on every data input, `workload(param = [rates])` runs the check once per rate and shows the verdict per rate, and `round = ...` replaces the generated workload entirely. The compaction store showed that the composition of the input can decide the verdict: a reserve of eight compacts everything under one put per four gets and falls behind under three puts per two gets. Nothing in the tool decides the operating point, and the owner has held that the checker should find amplification through scheduling alone under a fixed input, with load belonging to the test harnesses. The first step is to record, for every configuration in the three sets, whether its verdict changes across the rates in a small list, so the next spike knows how often the operating point matters in practice before deciding whether the tool may vary it.

4e. Operator-level counting

The counts see records entering steps and messages crossing the network, and nothing that happens inside a step: work performed as arithmetic inside an operator, or as a real scan that joins against state and produces records that never cross a hook, is invisible. The one intra-step count tried, DFIR handoff items, also counts the replay of persisted state into every step, and on that count four of the seven benign corpus configurations read hazardous; separating replay from newly produced records would need an operator-level count built from the operator graph. The owner decided not to build that until an existing witness needs it, and none does, because the compaction store models its scan rather than performing it. The first step is to rewrite the compaction witness so that a read actually joins against the uncompact segment and produces one record per record scanned; if the checker then cannot see the scan, that is the evidence that justifies the operator-level count. A related question is unruly: whether state re-read while a step waits, proportional to state size times delay with no feedback, counts as amplification; if it did, several assurance arguments would be wrong.

4f. Smaller items

The time-refilled budget's refill of four tokens per tick against two requests per round never emptied the bucket, so the prediction that a binding time budget grows with a smaller slope than no governance is untested; a configuration with a refill below the retry demand would test it. Snapshot holds are implemented and every snapshot hold in the corpus reads zero, but no program in any set makes its sends depend on a snapshot, so the snapshot hold has not met a case that could fail; a program that judges retries against a snapshot clock, or fetches because a snapshot cache still shows a miss, would be the first. Decision points are now named by the line of their own `use::batch` or `use::snapshot` call, and the position index within a step appears only when two points share a line, which no program in the three sets does, so the tie has not been exercised. The full-state gossip `g_set_gossip` in `hydro_std` cannot take the attribute because it returns a `Singleton`, which the harness has no way to sink, and its crate does not enable the simulator outside its tests; its hand-wired check exists in `checker_experiments.rs`.

5. How to run everything

All commands run from the repository root. The environment variable `SDKROOT=/Library/Developer/CommandLineTools/SDKs/MacOSX10.15.sdk` may be needed for the build on this machine, and the script exports it when the path exists.

To check one function with every configuration its attributes declare, and print each report and a summary table:

```
./cargo-check-amplification hydro_test rpc_with_retries
```

This takes about two minutes including the build. The two `rpc_with_retries` configurations run at the default horizon in about a minute after the build.

To check every annotated function in the crate:

```
./cargo-check-amplification --threads 4 hydro_test
```

This runs twenty-seven configurations in about eleven minutes on four threads. The blind program `batch_flush` is marked ignored in its attribute because its check takes about twenty minutes at the default horizon; `--include-ignored` runs it too.

The hand-wired validation tests, which assert the recorded verdict and named decision point for every configuration, run under `cargo test`:

```
cargo test -p hydro_test --lib library_check -- --nocapture --test-threads=1
cargo test -p hydro_test --lib blind_check -- --nocapture --test-threads=1
cargo test -p hydro_test --lib blind_budget_check -- --nocapture --test-threads=1
```

The first covers the twenty-two corpus configurations and takes about twenty-five minutes at the default horizon; the second covers the seven blind programs; the third covers the five retry-governance configurations

and takes about two minutes of test time. Each prints its full reports. A run at a horizon of 240 rounds, by changing `CheckConfig::new(240)` where the tests construct the config, takes about two minutes for all twenty-nine of the first two sets and is enough to see every verdict, since no verdict differs between 240 and 1000.

The witness programs' own labeling tests, which establish the ground truth independently of the checker, run with `cargo test -p hydro_test --lib witnesses::` (about ninety seconds of test time for the original corpus) and `cargo test -p hydro_test --lib rpc_retry::sim_tests`.